

## CAHIER DES CHARGES

# OpenEduVerse

Plateforme d'apprentissage en ligne — gestion de cours, quiz, paiements mobile money, messagerie et suivi de progression pour étudiants, enseignants et administrateurs.

---

| Projet                      | Stack                       | Date           |
|-----------------------------|-----------------------------|----------------|
| PPE — OpenEduVerse (ppe301) | Django · PostgreSQL · React | Septembre 2026 |

# Sommaire

---

## 1. Présentation générale du projet

---

## 2. Architecture technique

---

## 3. Gestion des utilisateurs et des rôles

---

## 4. Authentification et sécurité de connexion

---

## 5. Module Cours & contenu pédagogique

---

## 6. Module Quiz & évaluations

---

## 7. Module Paiement & portefeuille

---

## 8. Module Bibliothèque

---

## 9. Module Messagerie

---

## 10. Module Visioconférence

---

## 11. Notifications & suivi d'activité

---

## 12. Tableaux de bord par rôle

---

## 13. Pages publiques & refonte React

---

## 14. Identité visuelle & design system

---

## 15. Modèle de données complet

---

## 16. Infrastructure, environnement & déploiement

---

## 17. Limites connues & pistes d'évolution

---

# 1. Présentation générale du projet

---

OpenEduVerse est une plateforme d'e-learning complète, développée avec le framework Django, destinée à connecter trois catégories d'utilisateurs : **étudiants**, **enseignants** et **administrateurs**, autour de contenus pédagogiques, d'évaluations et d'un système de paiement mobile money adapté au contexte togolais.

Le projet couvre l'ensemble du cycle de vie d'une formation en ligne : découverte publique des cours, inscription, apprentissage (chapitres, ressources, vidéos), évaluation (quiz), certification, communication (messagerie, visioconférence) et monétisation (cours et chapitres premium, achats de livres, recharge de portefeuille via PayGate Global — TMoney/Flooz).

## 1.1 Objectifs

- Offrir un catalogue de formations structuré (catégories → cours → modules → chapitres) avec un modèle économique freemium (contenu gratuit et contenu premium payant, à la granularité du cours, du module ou du chapitre).
- Permettre aux enseignants de créer et gérer leurs cours, chapitres, ressources et quiz, avec correction manuelle ou assistée par intelligence artificielle.
- Permettre aux étudiants de suivre leur progression, passer des quiz, consulter leurs résultats et échanger avec les enseignants.
- Intégrer un moyen de paiement mobile money réel (PayGate Global) adapté au marché togolais, avec un mode simulation pour le développement.
- Fournir aux administrateurs des outils de pilotage : gestion des utilisateurs, des cours, des paiements et suivi de l'activité de la plateforme.
- Présenter une vitrine publique moderne et professionnelle, distincte de l'espace applicatif connecté.

## 1.2 Public cible

### ÉTUDIANT

S'inscrit, suit des cours gratuits ou premium, passe des quiz, consulte ses résultats, échange avec les enseignants, gère un portefeuille et achète du contenu.

### ENSEIGNANT

Crée et gère ses cours, modules, chapitres et ressources ; conçoit des quiz (manuellement ou via IA) ; corrige les soumissions ; anime des sessions de visioconférence.

### ADMINISTRATEUR

Supervise l'ensemble de la plateforme : utilisateurs, cours, paiements, activité, messages de contact reçus depuis le site public.

## 2. Architecture technique

### 2.1 Vue d'ensemble

La plateforme repose sur une architecture **hybride** :

- **Backend** : Django (application unique `eLearning`, projet `ppe301`), servant de source unique de vérité pour les données, l'authentification et la logique métier.
- **Base de données** : PostgreSQL.
- **Espace applicatif connecté** (tableaux de bord étudiant / enseignant / administrateur) : rendu côté serveur avec les gabarits Django classiques (HTML + Bootstrap/AdminLTE + jQuery), organisés autour d'un gabarit commun `admin/base_admin.html`.
- **Vitrine publique** (accueil, cours, à propos, contact, connexion, inscription) : reconstruite en **React** pour les quatre premières pages, avec une identité visuelle propre, tandis que connexion et inscription restent des gabarits Django serveur (formulaires sensibles), tous deux alignés sur la même charte graphique.

Le choix de ne pas migrer tout le site vers React a été délibéré : les ~90 vues du tableau de bord (paiements, quiz, messagerie...) restent inchangées afin de ne prendre aucun risque sur des fonctionnalités métier critiques déjà opérationnelles, pendant que la partie strictement publique bénéficie d'une modernisation complète.

### 2.2 Frontend public — détail technique

Les pages *Accueil*, *Cours*, *À propos* et *Contact* sont des mini-applications React indépendantes (architecture « multi-entrées »), compilées avec **Vite** :

| Élément                       | Détail                                                                                                                                                                            |
|-------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Emplacement du code source    | <code>frontend/src/pages/{home,courses,about,contact}</code>                                                                                                                      |
| Composants partagés           | <code>frontend/src/shared/</code> : <code>Nav.jsx</code> , <code>Footer.jsx</code> , <code>api.js</code> (fetch + cookie CSRF), <code>branding.css</code> (design tokens)         |
| Build                         | Sortie fixe (noms de fichiers non hashés) dans <code>eLearning/static/react_public/</code> , chargée par des gabarits Django minimalistes ( <code>templates/react/*.html</code> ) |
| Communication avec le backend | API JSON dédiée <code>eLearning/api_public.py</code> , sans dépendance ajoutée (pas de Django REST Framework), cohérente avec le style déjà utilisé ailleurs dans le projet       |

Chaque page Django ne fait plus que servir une coquille HTML minimale (`<div id="react-root">` + scripts/styles compilés) ; React prend le relais côté navigateur pour l'affichage et les appels réseau. Les formulaires sensibles (achat de cours, inscription à un cours gratuit) restent de **vrais formulaires HTML** soumis directement à Django (et non des appels asynchrones), afin de préserver le fonctionnement existant des messages flash et des redirections après paiement.

## 2.3 Environnement de développement

| Composant                | Version / détail                                                                                                            |
|--------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| Python / Django          | Python 3.14, Django 6.0                                                                                                     |
| Base de données          | PostgreSQL (base <code>elearning</code> )                                                                                   |
| Node.js (build frontend) | Node 20 (installation portable dans <code>.tools/</code> , sans modification système)                                       |
| Bundler                  | Vite 5 + plugin React                                                                                                       |
| Serveur de développement | <code>python manage.py runserver</code> , exposé publiquement via un tunnel <b>ngrok</b> pour les tests de paiement PayGate |

## 3. Gestion des utilisateurs et des rôles

### 3.1 Modèle de compte

Le modèle `User` personnalisé (authentification par email, pas de nom d'utilisateur) porte les informations communes à tous les comptes. Trois sous-modèles héritent de `User` par héritage multi-tables et portent des informations spécifiques :

| Champ                                  | Description                                                                   |
|----------------------------------------|-------------------------------------------------------------------------------|
| <code>nom</code> , <code>prenom</code> | Identité de l'utilisateur                                                     |
| <code>email</code>                     | Identifiant de connexion, unique                                              |
| <code>sexe</code>                      | Homme / Femme                                                                 |
| <code>date_naissance</code>            | Date de naissance                                                             |
| <code>telephone</code>                 | Numéro de téléphone (réservé à l'usage futur d'un canal SMS)                  |
| <code>adresse</code>                   | Adresse postale                                                               |
| <code>role</code>                      | <code>etudiant</code> / <code>enseignant</code> / <code>administrateur</code> |
| <code>photo_profil</code>              | Photo de profil                                                               |
| <code>theme</code>                     | Préférence d'affichage clair/sombre du tableau de bord                        |

#### ETUDIANT

`niveau` , `etablissement` , `solde` (portefeuille pour les achats)

#### ENSEIGNANT

`specialite`

### 3.2 Parcours d'inscription

Formulaire public ( `/register/` ) : nom, prénom, sexe, date de naissance, email, téléphone, adresse, mot de passe (+ confirmation), rôle souhaité (étudiant ou enseignant), et — pour les étudiants — une sélection de **centres d'intérêt** (matières) qui génère automatiquement des inscriptions aux cours correspondants.

À l'issue de l'inscription, le compte est créé mais **aucune connexion automatique n'est effectuée** : l'utilisateur est redirigé vers la page de connexion pour s'authentifier lui-même.

## 4. Authentification et sécurité de connexion

La connexion combine mot de passe et double authentification par email, en deux étapes :

### 4.1 Étape 1 — Email + mot de passe

`/dashboard/admin/login/` : l'utilisateur saisit son email et son mot de passe. En cas de succès, un code de sécurité à 6 chiffres est généré et envoyé par email.

### 4.2 Étape 2 — Vérification du code

`/dashboard/admin/login/code/` : l'utilisateur saisit le code reçu.

- Code valable **10 minutes**.
- **5 tentatives** maximum avant expiration forcée du processus.
- Bouton de renvoi du code en cas de non-réception.
- Le code et son expiration sont stockés côté session serveur (aucune persistance en base), effacés dès la connexion réussie ou l'expiration.

Une fois validé, l'utilisateur est redirigé automatiquement vers le tableau de bord correspondant à son rôle (étudiant, enseignant ou administrateur) — **le même formulaire de connexion est utilisé par tous les rôles**, y compris les administrateurs ; seule la redirection finale diffère.

### 4.3 Envoi de l'email

L'envoi utilise `send_mail` de Django. Deux modes selon la configuration :

| Mode                    | Condition                                                                  | Comportement                                                                                 |
|-------------------------|----------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| Développement (console) | Aucun identifiant SMTP dans <code>.env</code>                              | Le code est affiché directement sur la page de vérification (bandeau « mode développement ») |
| Production (SMTP réel)  | <code>EMAIL_HOST_USER</code> / <code>EMAIL_HOST_PASSWORD</code> renseignés | Envoi réel via Gmail SMTP (mot de passe d'application)                                       |

### 4.4 Autres mesures de sécurité

- Protection CSRF sur tous les formulaires et endpoints API.
- Champs de formulaire avec `autocomplete="off"` (et `new-password` pour les mots de passe) afin d'éviter la pré-suggestion de valeurs par le navigateur.
- Vérification serveur des paiements PayGate (le webhook ne fait jamais confiance aux données reçues : il réinterroge systématiquement l'API PayGate avant de créditer un compte), avec verrouillage en base (`select_for_update`) pour éviter les doubles crédits en cas de requêtes concurrentes.
- Journalisation des actions sensibles (connexions, inscriptions) dans le modèle `LogActivite`, avec adresse IP.

## 5. Module Cours & contenu pédagogique

---

### 5.1 Structure hiérarchique

`Categorie` (avec sous-catégories) → `Cours` → `Module` → `Chapitre` → `RessourceChapitre`, avec en complément des `RessourceCours` attachées directement au cours.

### 5.2 Modèle économique premium

Le contenu payant peut être défini à **trois niveaux** indépendants :

- **Cours entier** premium ( `Cours.est_premium` + `prix` )
- **Module** premium ( `Module.est_premium` )
- **Chapitre** premium ( `Chapitre.est_premium` + `prix` propre )

Un étudiant achète l'accès via son portefeuille (solde) ou directement via PayGate ; l'accès est tracé par `ChapitreDebloque` (achat) tandis que `ChapitreVu` trace la simple consultation, y compris de contenu gratuit — ces deux notions sont volontairement distinctes pour permettre un futur calcul de progression fiable.

### 5.3 Gestion enseignant

Création de cours complets ( `creer_cours_complet` ), gestion des chapitres, modules et ressources associées, bascule premium/gratuit à la volée, upload de vidéos (lien YouTube) et de fichiers ressources (PDF, images, etc. — avec détection automatique de l'icône par extension).



## 6. Module Quiz & évaluations

---

| Entité         | Rôle                                                                                                                 |
|----------------|----------------------------------------------------------------------------------------------------------------------|
| Quiz           | Titre, durée, note maximale, type de correction (manuelle ou IA), rattaché à un cours et optionnellement à un module |
| Question       | Texte, média joint, points, ouverte ou à choix                                                                       |
| Choix          | Options de réponse (QCM), avec indicateur de bonne réponse                                                           |
| SoumissionQuiz | Réponse d'un étudiant (fichier ou réponses QCM en JSON), état de correction, note, commentaires                      |

### 6.1 Génération assistée par IA

Les enseignants peuvent générer un quiz automatiquement à partir d'un sujet via l'API **Google Gemini** (appel REST direct, sans dépendance lourde), produisant un jeu de questions structuré au format JSON directement exploitable.

### 6.2 Correction

Deux modes : **correction manuelle** par l'enseignant (file d'attente des soumissions en attente) ou **correction automatique par IA** pour les quiz configurés en conséquence.

## 7. Module Paiement & portefeuille

---

### 7.1 Portefeuille étudiant

Chaque étudiant dispose d'un solde ( `Etudiant.solde` ) rechargeable, utilisable pour acheter des cours premium, des chapitres premium ou des livres.

### 7.2 PayGate Global (TMoney / Flooz)

Intégration réelle avec l'API de paiement mobile money togolaise PayGate Global :

- `PaielementPayGate` trace chaque tentative (identifiant unique, montant, type — recharge ou achat de cours —, statut, référence retournée par PayGate).
- Le paiement peut être confirmé par **webhook** ( `paygate_callback` ) ou par **polling côté client** ( `paygate_statut_json` ) : dans les deux cas, le serveur vérifie toujours l'état réel auprès de l'API PayGate avant de créditer quoi que ce soit.
- Un **mode simulation** complet ( `paygate_simulation*` ) permet de tester le parcours de paiement en développement, avec un OTP simulé envoyé par SMS via l'API gratuite Textbelt (1 SMS/jour, non adapté à une mise en production).

### 7.3 Historique

`TransactionSimulee` et `Paielement` conservent l'historique des opérations (recharge, achat de cours, achat de chapitre, achat de livre) consultable par l'étudiant et par l'administrateur.

## 8. Module Bibliothèque

---

Catalogue de livres ( [Livre](#) ) avec couverture, description, prix, statut premium. Les étudiants achètent un accès ( [AchatLivre](#) ) débité de leur portefeuille, consultable ensuite depuis leur espace.

## 9. Module Messagerie

---

Messagerie interne entre utilisateurs ( **Conversation** à participants multiples, **Message** horodatés et marqués lus/non lus). L'affichage des nouveaux messages utilise un mécanisme de **polling AJAX** (rafraîchissement périodique côté client) pour une expérience quasi temps réel sans recourir aux WebSockets.

## 10. Module Visioconférence

---

Les enseignants créent des sessions ( [SessionVisio](#) : titre, date, durée, lien de réunion externe type Google Meet) rattachées à un cours, visibles par les étudiants inscrits.

## 11. Notifications & suivi d'activité

---

**Notification** alerte les utilisateurs (nouveau cours dans leur domaine, nouvelle ressource, quiz corrigé).

**LogActivite** journalise les actions importantes (connexions, inscriptions) avec horodatage et adresse IP, consultable par l'administrateur via le tableau de suivi d'activité.

## 12. Tableaux de bord par rôle

---

### Étudiant

- Mes cours, mes chapitres débloqués, progression
- Passage de quiz et consultation des résultats
- Portefeuille, recharge, historique d'achats
- Bibliothèque de livres
- Messagerie

### Enseignant

- Gestion complète des cours, modules, chapitres et ressources
- Création et suivi des quiz, file de corrections en attente
- Planification des sessions de visioconférence
- Messagerie

### Administrateur

- Gestion des utilisateurs et des cours (vue globale)
- Suivi des paiements et de l'activité de la plateforme
- Consultation des messages reçus via le formulaire de contact public

## 13. Pages publiques & refonte React

---

Les quatre pages publiques ont été entièrement reconstruites, avec pour chacune un rôle précis :

### 13.1 Accueil

Section d'en-tête avec message d'accroche, présentation courte de la plateforme, appel à l'action vers le catalogue de cours ou l'inscription (selon l'état de connexion de l'utilisateur, déterminé via l'API de session).

### 13.2 Cours

Catalogue paginé (6 cours par page) consommant l'API `/api/public/courses/`, avec pour chaque cours : image, titre, nom réel de l'enseignant, badge premium/gratuit, et un bouton d'action qui s'adapte à quatre situations : visiteur non connecté (lien vers la connexion), utilisateur connecté non étudiant (lien vers son tableau de bord), étudiant déjà inscrit (lien direct vers le cours), étudiant non inscrit (formulaire d'achat ou d'inscription gratuite, soumis directement à Django).

### 13.3 À propos

Présentation de la mission de la plateforme et de ses trois piliers (innovation, communauté, excellence).

### 13.4 Contact

Formulaire (nom, email, message) soumis en JSON à l'API `/api/public/contact/`, avec validation et retour d'erreur inline sans rechargement de page. Chaque soumission crée un enregistrement `ContactMessage` consultable dans l'interface d'administration Django.

### 13.5 Connexion & Inscription

Restent des gabarits Django server-rendered (formulaires sensibles), mais entièrement réhabillés avec la même identité visuelle que les pages React, via une feuille de style dédiée (`auth_public.css`) et les mêmes composants de navigation/pied de page reproduits en gabarits Django.



## 14. Identité visuelle & design system

### 14.1 Palette

La palette a été dérivée directement du logo officiel OpenEduVerse (globe bleu marine surmonté d'un chapeau de diplômé, mot-symbole bicolore) plutôt que d'une couleur arbitraire :

| Rôle                     | Couleur                                                                                   |
|--------------------------|-------------------------------------------------------------------------------------------|
| Encre / texte principal  | Bleu marine <code>#142a4f</code>                                                          |
| Accent principal         | Orange <code>#f5a623</code> (issu du mot « Open » du logo)                                |
| Fond de page             | Blanc cassé neutre <code>#f8f7f4</code>                                                   |
| Accent secondaire (rare) | Rouge <code>#e1442d</code> , repris du trait du logo, utilisé en petite touche décorative |

### 14.2 Typographies

- **Baloo 2** — police d'affichage arrondie pour les titres, en écho au style de lettrage du logo
- **Manrope** — police pour le texte courant

### 14.3 Composants

Navigation et pied de page unifiés (logo + liens + boutons de connexion/inscription en pilule), cartes de cours, badges de statut, formulaires épurés — reproduits à l'identique entre les pages React et les pages Django classiques (connexion/inscription) pour une continuité visuelle totale.

Le design a fait l'objet de plusieurs itérations validées avec le commanditaire du projet, en évitant délibérément les motifs visuels perçus comme génériques ou « produits par une IA » (dégradés teal/vert, glassmorphism, cartes icône-dans-un-rond, boutons pilule systématiques) au profit de choix ancrés dans l'identité de marque existante.

## 15. Modèle de données complet

| Modèle                                        | Rôle                                                     |
|-----------------------------------------------|----------------------------------------------------------|
| User / Etudiant / Enseignant / Administrateur | Comptes utilisateurs et rôles (héritage multi-tables)    |
| Categorie                                     | Catégories de cours, hiérarchiques                       |
| Cours                                         | Formation proposée par un enseignant                     |
| Module                                        | Regroupement de chapitres au sein d'un cours             |
| Chapitre                                      | Unité de contenu (texte, ressource, vidéo)               |
| RessourceCours / RessourceChapitre            | Fichiers annexes attachés à un cours ou un chapitre      |
| ChapitreDebloque                              | Trace un achat d'accès à un chapitre premium             |
| ChapitreVu                                    | Trace la consultation d'un chapitre (progression)        |
| Inscription                                   | Inscription d'un étudiant à un cours                     |
| Quiz / Question / Choix                       | Structure d'un quiz et de ses questions                  |
| SoumissionQuiz                                | Réponse et correction d'un étudiant à un quiz            |
| Paielement                                    | Paielement enregistré (historique)                       |
| PaielementPayGate                             | Suivi détaillé d'un paiement mobile money réel           |
| TransactionSimulee                            | Historique des mouvements du portefeuille étudiant       |
| SessionVisio                                  | Session de visioconférence planifiée                     |
| Conversation / Message                        | Messagerie interne                                       |
| Notification                                  | Alertes utilisateur                                      |
| Livre / AchatLivre                            | Bibliothèque et achats de livres                         |
| ContactMessage                                | Message reçu via le formulaire de contact public         |
| LogActivite                                   | Journal des actions sensibles (connexions, inscriptions) |

## 16. Infrastructure, environnement & déploiement

---

### 16.1 Variables d'environnement ( `.env` )

| Variable                                                        | Usage                                                    |
|-----------------------------------------------------------------|----------------------------------------------------------|
| <code>GEMINI_API_KEY</code>                                     | Génération de quiz par IA                                |
| <code>PAYGATE_API_KEY</code>                                    | Paielements mobile money réels                           |
| <code>PAYGATE_SIMULATION</code>                                 | Force le mode simulation de paiement                     |
| <code>SITE_BASE_URL</code>                                      | URL publique (tunnel ngrok) pour les retours de paiement |
| <code>EMAIL_HOST_USER</code> / <code>EMAIL_HOST_PASSWORD</code> | Envoi réel des codes de connexion par email (SMTP Gmail) |

Ce fichier n'est jamais commité dans le dépôt (exclu via `.gitignore`) ; en son absence, chaque fonctionnalité bascule vers un mode dégradé sûr (simulation de paiement, code affiché à l'écran plutôt qu'envoyé par email, etc.) plutôt que de provoquer une erreur.

### 16.2 Fichiers statiques et médias

Les fichiers statiques (CSS, JS, images, bundle React compilé) sont servis depuis `eLearning/static/` ; les fichiers déposés par les utilisateurs (photos de profil, images de cours, ressources, livres) depuis `media/`.

## 17. Limites connues & pistes d'évolution

---

- Le canal SMS pour la double authentification n'est pas encore actif : le numéro de téléphone est collecté à l'inscription mais aucun fournisseur SMS payant n'est pour l'instant intégré (l'offre gratuite disponible dans le projet — 1 SMS/jour — ne permet pas un usage multi-utilisateurs).
- La migration vers React ne couvre pour l'instant que les pages publiques et les écrans de connexion/inscription ; les tableaux de bord restent sur l'architecture Django classique.
- Aucune suite de tests automatisés n'est en place à ce stade — la validation repose sur des vérifications manuelles et fonctionnelles ciblées.